

Lab 04 — Vragen

Vraag 1 [Diagnose]

Een ontwikkelaar voert `podman build -t api:1.0 .` uit vanuit `~/projecten/api/`. Zijn Dockerfile bevat `COPY ../gemeenschappelijk/config.yml /app/`, en de build mislukt met `possible escaping context directory error`. Leg uit waarom, en waarom `-f`, een absoluut pad of `sudo` geen van alle helpen. Wat is de correcte oplossing?

Vraag 2 [Analyse]

Onder Docker duurt de build van een Angular-project 4 minuten, waarvan 3 min 20 s als `transferring context` in beeld staan. De projectmap bevat `node_modules/` (900 MB) en `.git/` (200 MB). Een collega stapt over op Podman, ziet de build terugvallen tot 40 seconden en concludeert dat `.dockerignore` niet meer nodig is. Leg uit wat Docker deed, wat er met Podman veranderd is, en waarom je collega zich vergist — benoem daarbij het **tweede** risico, dat niets met traagheid te maken heeft.

Vraag 3 [Begrip]

Een Dockerfile bevat `EXPOSE 8080`. De ontwikkelaar start `podman run -d mijn-api:1.0`, merkt dat `curl http://localhost:8080` geen antwoord geeft, en concludeert dat de image kapot is. Klopt die conclusie? Leg uit wat `EXPOSE` precies doet.

Vraag 4 [Analyse]

Vergelijk deze twee Dockerfiles. Wat doet elk precies, en welke is correct voor een API-image?

```
# A
FROM docker.io/library/eclipse-temurin:21-jre
COPY api.jar /app/api.jar
RUN java -jar /app/api.jar
```

```
# B
FROM docker.io/library/eclipse-temurin:21-jre
COPY api.jar /app/api.jar
CMD ["java", "-jar", "/app/api.jar"]
```

Vraag 5 [Analyse]

Hier zijn twee images. Zeg voor elk wat `podman run img` en `podman run img --debug` opleveren.

```
# A
ENTRYPOINT ["java", "-jar", "/app/api.jar"]
CMD ["--spring.profiles.active=prod"]
```

```
# B
CMD ["java", "-jar", "/app/api.jar", "--spring.profiles.active=prod"]
```

Zeg vervolgens bij welke van de twee je nog `podman run img sh` kunt gebruiken om te debuggen, en hoe je bij de andere alsnog aan een shell raakt.

Vraag 6 [Diagnose]

Een team klaagt dat elke heruitrol tien seconden langer duurt dan verwacht, dat Podman telkens `resorting to SIGKILL` toont, en dat Spring Boot zijn *shutdown hooks* nooit uitvoert. De Dockerfile eindigt met:

```
CMD java -jar /app/api.jar
```

Stel de diagnose, geef de correctie, en leg uit waarom die ene regel volstaat om het symptoom te veroorzaken — en waarom het team er **niets** van merkt op zijn testimage op basis van Alpine.

Vraag 7 [Analyse]

Een Dockerfile heeft bij het opstarten de variabele `JAVA_OPTS` nodig:

```
ENV JAVA_OPTS="-Xmx512m"  
ENTRYPOINT ["java", "$JAVA_OPTS", "-jar", "/app/api.jar"]
```

De container mislukt met `Unrecognized option: $JAVA_OPTS`. Leg uit wat er fout gaat, en geef **twee** mogelijke correcties, telkens met wat ze kosten.

Vraag 8 [Begrip]

Een ontwikkelaar geeft het databasewachtwoord mee bij de build — `podman build --build-arg DB_PASSWORD=Secr3t! -t api:1.0 .` — met als argument dat een `ARG` niet in de image achterblijft. Is het wachtwoord veilig? Onderbouw je antwoord en geef het commando dat het bewijst.

Vraag 9 [Analyse]

Een Maven-Dockerfile ziet er zo uit:

```
COPY . /app  
WORKDIR /app  
RUN mvn -q package -DskipTests
```

Elke build duurt 6 minuten, zelfs wanneer er maar één `.java`-bestand veranderd is. Herschrijf de instructies in de juiste volgorde, leg het cachemechanisme uit dat jouw versie sneller maakt, en zeg welke build ondanks alles traag zal blijven.

Vraag 10 [Diagnose]

```
RUN apt-get update  
RUN apt-get install -y curl vim  
RUN rm -rf /var/lib/apt/lists/*
```

Wijs **drie** verschillende gebreken in deze drie regels aan, en geef de correcte versie.

Vraag 11 [Analyse]

Een ontwikkelaar wijzigt alleen de laatste `COPY` van zijn Dockerfile en ziet de build `--> Using cache` tonen voor de eerste acht stappen, waarna de laatste twee herbouwd worden. De dag erna voegt hij op de **derde** positie een `ENV`-variabele toe, en de hele build begint van nul. Verklaar beide gedragingen met één en dezelfde regel.

Vraag 12 [Begrip]

`COPY` en `ADD` lijken inwisselbaar. Geef twee gedragingen die eigen zijn aan `ADD`, leg uit waarom de officiële aanbeveling `COPY` is, en noem het enige geval waarin `ADD` nog te verantwoorden valt.

Vraag 13 [Analyse]

Een Dockerfile zet `USER 1000:1000` meteen na de `FROM`, vóór de instructies `COPY` en `RUN apt-get install`. De build mislukt met `Permission denied`. Leg uit waarom, en zeg waar `USER` thuishoort in een goed geschreven Dockerfile. En dan: in rootless-modus toont `podman top` voor die container `USER 1000` en `HUSER 100999`. Waar komt dat tweede getal vandaan, en waarvoor dient `USER` nog, als "root" toch al geen echte root is?

Vraag 14 [Analyse]

Een collega beweert: "Je moet het aantal lagen zo klein mogelijk houden, dus alles in één enkele `RUN`." Bespreek: wanneer heeft hij gelijk, en wanneer gaat die regel ten koste van de buildtijd en de omvang van de overdrachten?